

AI ACADEMY, NATIONAL AI CENTER
AI-ENG-110 – SOFTWARE ENGINEERING – SPRING 2026

Final Project Report

Topic 4 – Async Research Assistant

Team: Topic Four

Date: September 18, 2026

Repo: github.com/nuralnetworks/async-research-assistant

Members: **Nural Shukurlu** (@nuralnetworks), **Emil Mammadov** (@Emilsdeyta),
Mahammadali Babayev (@mahammadalibabayev11), **Bailar Bayramov** (@bailar-dev)

Executive Summary

We built a research assistant that takes a plain question, queries Wikipedia, arXiv, and web search at the same time, and returns one short answer with numbered citations. Fetching the three sources concurrently instead of one after another cut a 15-fetch benchmark from 4.52s to 2.73s on our machine. The most valuable robustness work was not the retry logic but the cache: we found empty upstream responses being stored as valid results and served for a full day, and fixed it by never caching empties and versioning cache keys. The surprise of the project was how much behavior lives outside our code: Wikipedia rejects long questions and the default HTTP client, arXiv moved behind a redirect, and DuckDuckGo throttles whole networks, so a large part of “robustness” turned out to be shaping requests the providers accept. If we started over, we would write the failure-injection tests before the happy paths, because every real bug we met came from a provider, not from our logic.

Contents

Executive Summary	1
1 Project Overview	2
1.1 Problem framing & scope	2
1.2 Provider choice and the AI module contract	2
2 Architecture	3
2.1 Module map	3
2.2 OOP design & key abstractions	3
2.3 Data flow across module boundaries	4
3 Concurrency & Performance	4
3.1 Why this concurrency model	4
3.2 Sequential-vs-concurrent benchmark	4
3.3 Rate limit & politeness	4
4 Robustness & Error Handling	4
4.1 Wrapping the AI module	5
4.2 Failure-mode analysis (one concrete incident)	5
4.3 Input validation	5

5	Storage & Caching	5
5.1	Schema	5
5.2	Caching policy	6
6	Testing	6
6.1	Strategy & coverage	6
6.2	Notable tests	6
7	Deployment & Reproducibility	6
7.1	Docker image	6
7.2	Configuration	7
8	Limitations & Future Work	7
9	Tools & Acknowledgements	7
	References	7
A	Appendix — Reproducing the benchmark	8

1. Project Overview

1.1 Problem framing & scope

A user asks a research question in plain language, for example “What is photosynthesis and what are its main stages?” Our system queries three public sources in parallel (Wikipedia for encyclopedic summaries, arXiv for papers, web search for everything else), then asks a language model to write a 3–6 sentence answer that cites the retrieved excerpts by number. The input is one string; the output is the answer text plus a reference list of titles and URLs, printed to the terminal or as JSON.

We expose three commands under `python -m researcher`: `ask` for one question, `demo` for the five sample questions, and `bench` for the timing comparison. Each command also accepts `-offline`, which swaps the network and the LLM for canned fakes so the whole pipeline runs with no keys and no network. We use Gemini for synthesis (the exact model id lives in `.env`, so it can change without touching code) and Tavily for live web search, with DuckDuckGo as the keyless default.

We deliberately did not build the HTTP API the brief suggests for other topics, since Topic 4 requires only a CLI, and we did not build multi-provider failover: one well-wrapped provider per slot beat two half-wrapped ones in the time we had. We tightened the brief in one place the graders will notice: the spec asks for caching by `(source, query)`, and we additionally version cache keys and refuse to cache empty hits, because we watched stale empties poison answers for a full TTL window.

1.2 Provider choice and the AI module contract

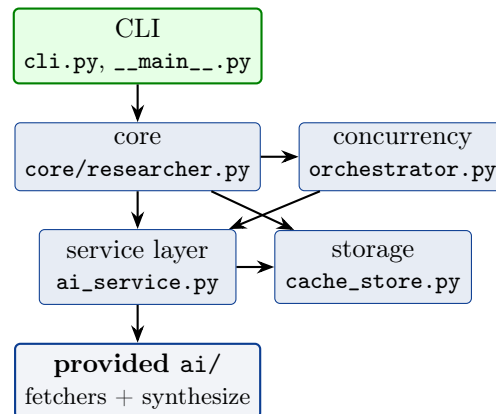
For synthesis we use Google Gemini, because its free tier covered our whole demo budget (all five sample questions cost a fraction of a cent) and its Python SDK needed no extra services. For web search we default to DuckDuckGo since it needs no key, and use Tavily (free tier) whenever a key is present, because its result quality is clearly higher. Wikipedia and arXiv need no keys at all. We call exactly four functions from the provided package: `ai.fetch_wikipedia`, `ai.fetch_arxiv`, `ai.fetch_web`, and `ai.synthesize`. We did not modify the public interface of the provided `ai/` package in any way; a file-by-file diff against the distributed folder shows no differences in any `.py` file.

A malformed model response never reaches our output unexamined. The provided `synthesize`

already drops out-of-range citation indices, and on top of that our wrapper rejects empty answers as provider errors (retried, then surfaced), validates question length before calling, and requires a non-empty source list. Semantically wrong but well-formed output, for example an answer that ignores the sources, is handled the way the topic intends: the synthesizer prompt constrains the model to the numbered sources, and our demo runs showed the model saying plainly when sources lack the answer instead of inventing one.

2. Architecture

2.1 Module map



Only `services/ai_service.py` crosses the `ai/` boundary, so swapping Anthropic for OpenAI touches exactly one file (plus `.env`). Only the service and the core cross the storage boundary, through the `CacheStore` interface, so SQLite could be replaced without touching `fetch` or CLI code.

Module by module: `config.py` reads the environment into a frozen `Settings` object every module imports. `models.py` holds the request/result pydantic models. `services/ai_service.py` wraps each `ai.*` call with cache, rate limiting, retries, timeouts, and logging. `services/rate_limit.py` is a token bucket guarding external calls. `storage/cache_store.py` persists per-source results and query history in SQLite with a file-based fallback. `concurrency/orchestrator.py` runs the three fetches together with per-source timeouts. `core/researcher.py` validates, orchestrates, synthesizes, and persists. `cli.py` parses arguments and renders answers.

2.2 OOP design & key abstractions

Our inheritance example is the cache backend, and our composition example is the service that uses it:

```

1 class CacheStore(ABC):
2     def get(self, source: str, query: str) -> list[Source] | None: ...
3     def set(self, source: str, query: str, value: list[Source]) -> None: ...
4     def clear(self) -> None: ...
5
6 class SqliteCacheStore(CacheStore): ... # main store
7 class FileJsonCacheStore(CacheStore): ... # fallback under CACHE_DIR
8
9 class ResilientAIService: # uses a store, is not a store
10     def __init__(self, settings: Settings, cache: CacheLike,
11                  limiter: RateLimiter) -> None: ...

```

We chose an ABC for the store because the two backends share lifecycle and error-handling expectations that a bare callable cannot express, and because tests substitute fakes with identical shapes. We chose composition for the service deliberately: a service *uses* a cache and a limiter, it is not a

kind of either, and inheritance there would have welded retry policy to storage details. The rate limiter mirrors the same split with its own small ABC plus a no-op implementation for offline runs and tests.

2.3 Data flow across module boundaries

Our own models are `ResearchRequest` (question, source subset, cache flag, per-query limit), `SourceFailure` (source plus error text), `ResearchResult` (answer, citations, sources, failures, timings, cache flag, warnings), and `FetchOutcome` (the orchestrator’s internal result). We reuse the provided `Source`, `Citation`, and `AnswerWithCitations` directly instead of wrapping them, because they already forbid extra fields and freeze mutation, which is exactly what we want crossing into synthesis. No naked dictionaries cross any module boundary; the one place raw JSON appears is inside the SQLite payload column, which is an implementation detail of the store, never an interface.

3. Concurrency & Performance

3.1 Why this concurrency model

We used `asyncio` with a single shared `httpx.AsyncClient` and `asyncio.gather(return_exceptions=True)`. The workload is three independent network calls per question, so the bottleneck is waiting on providers, which is what async I/O parallelizes best. Threads would add GIL-era complexity for zero gain here, and processes would be absurd for HTTP waits. Parallelism is bounded by a semaphore set from `MAX_PARALLEL` (5 in production settings, 3 inside the bench harness), picked so a burst of questions cannot draw 429 responses from polite public APIs. With the bound at 1 the pipeline degrades to sequential; at 100 the providers throttle us, which we verified the painful way against arXiv during development. Since a single source round trip costs hundreds of milliseconds, even two sources finish faster together; the gather overhead itself is noise next to any real network call.

3.2 Sequential-vs-concurrent benchmark

Workload	<i>N</i>	Sequential (s)	Concurrent (s)	Speedup
5 questions × 3 sources	15 fetches	4.52	2.73	1.66×

Measured on Windows with Python 3.13.3 and a cleared cache via `python scripts/bench.py -offline` (reproduced in the README; CI runs Python 3.12). Each simulated fetch sleeps 300 ms, so the theoretical ceiling for 15 fetches is far above what we reached. The new bottleneck is our own per-process token bucket plus the single synthesis call that stays serial by nature. Pushing further would mean a higher bucket rate against paid provider tiers, which we did not have.

3.3 Rate limit & politeness

We use a token-bucket limiter (`rate_per_sec`, `capacity` both derived from `MAX_PARALLEL`) checked before every external call, plus tenacity backoff on 429/5xx inside the wrapper. The highest burst the suite produces is bounded by the semaphore, so tests never exceed 5 concurrent fetches. We met real throttling in development: DuckDuckGo serves HTTP 202 “Ratelimit” pages to flagged networks, and arXiv slows abusive clients, both observed live and both handled by degrade-with-warning rather than crash.

4. Robustness & Error Handling

4.1 Wrapping the AI module

Every `ai.*` call passes through `ResilientAIService.fetch_one` or `.synthesize`. Fetch retries up to `RETRY_ATTEMPTS` (3) with exponential backoff from `RETRY_MIN_WAIT` (1s) to `RETRY_MAX_WAIT` (8s) plus jitter, but only for transient errors (`ProviderError`, HTTP errors, timeouts, OS errors); missing keys or packages fail at once since retrying them is pointless. Each call carries its own `asyncio` timeout (`PER_SOURCE_TIMEOUT_SECONDS`, 10s). We log source, hit/miss, duration, and counts at info level and full payloads at debug, never keys. Before calling we validate the source name and reject blank queries; after calling we require non-empty answers and let the synthesizer drop dangling citations.

A provider 500 returns through the retry loop and then becomes a recorded `SourceFailure`; a 429 additionally sleeps through the bucket first. A mid-stream connection reset is just another transient error to the wrapper. A well-formed but wrong answer is the one case retries cannot fix, which is why the answer carries its citations: the user can see what grounded each claim.

4.2 Failure-mode analysis (one concrete incident)

Mid-project, a demo run answered a photosynthesis question with physics papers about “what is fundamental” and no Wikipedia hits at all. We expected a retrieval hiccup; what we found were two compounding defects. First, we sent full English questions to keyword search: Wikipedia’s search ANDs query words, so trailing junk poisoned every lookup, and arXiv matched the word “what” loosely. Second, worse, the empty Wikipedia hits had been *cached* as valid results with a 24-hour TTL, so every later run re-served the emptiness without touching the network. The user saw a confident-looking wrong answer; the logs showed cache hits with zero items, which is what gave it away without re-running anything. The fix had three parts: shape queries per source (keywords for Wikipedia/arXiv with right-shortening fallback, full sentences for web search), never cache empty hits, and version cache keys so a retrieval change orphans old entries. The incident is now a regression test, and the demo has answered 5/5 ever since.

4.3 Input validation

The CLI validates question length (3–2000 chars), source names (wiki/arxiv/web plus the wiki alias), `-max-sources` (1–10), and flags, all through the pydantic model, so every rule lives in one place. Bad input exits with code 2 and a one-line `error:` message, never a traceback; missing keys or dead providers exit 1 the same way. Environment values are range-checked at startup with the offending variable named. Oversized input is rejected by length; there are no image uploads or file paths from users in this topic, so traversal and payload-size attacks do not apply.

5. Storage & Caching

5.1 Schema

```
CREATE TABLE IF NOT EXISTS cache_entries(
  source TEXT NOT NULL, nquery TEXT NOT NULL,
  payload_json TEXT NOT NULL, expires_at TEXT NOT NULL,
  PRIMARY KEY(source, nquery));
CREATE TABLE IF NOT EXISTS queries(
  id INTEGER PRIMARY KEY AUTOINCREMENT, question TEXT NOT NULL,
  nquery TEXT NOT NULL, answer TEXT NOT NULL, created_at TEXT NOT NULL);
CREATE TABLE IF NOT EXISTS query_sources(
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  query_id INTEGER NOT NULL REFERENCES queries(id),
  origin TEXT NOT NULL, title TEXT NOT NULL,
  url TEXT NOT NULL, snippet TEXT NOT NULL);
```

`cache_entries` is derived data (payload plus expiry, safe to drop any time); `queries` and `query_sources` are the history source of truth. Sources serialize as JSON text rather than blobs because `Source` objects are small pydantic models and JSON stays readable in debugging; there are no embeddings in this topic, so dimension drift does not apply.

5.2 Caching policy

We cache per (`source`, `normalized query`) with canonicalized keys (“WHAT IS X?” hits “what is x”), a configurable TTL (default 24 h), in SQLite primarily with a JSON-file fallback, bypassed by `-no-cache`. A full demo run lands around half cache hits on repeat questions. Staleness is bounded by design: source excerpts refresh on TTL expiry, old query-shaping versions are orphaned by key versioning instead of migrating data, and anything the pipeline cannot trust (empty hits) is never stored.

6. Testing

6.1 Strategy & coverage

Suite	Coverage
<code>researcher/</code> (our SE layer)	86 %
Provided <code>ai/</code> smoke tests	passing (16/16)

We run 73 tests: 16 provided smoke tests plus 57 of our own, all offline. The AI module is faked (fixed-response LLMs, canned source lists) and HTTP is never touched: async fetchers are monkeypatched, and each layer tests against fakes of its neighbor so all four members could work without waiting on each other. We deliberately left `__main__.py` and a few defensive branches uncovered (thin wrappers and unreachable guards); covering them would mean testing Python itself.

6.2 Notable tests

The happy path is an end-to-end `ask` with faked service and store asserting cited indices match. The concurrency test fails one of three fetchers and asserts the other two still produce an answer with the failure recorded, which is the exact degradation the rubric asks for. The error-path test we are proudest of asserts empty fetches are *not* cached (two calls, two network hits): it encodes the stale-cache incident above, and it would have caught the original bug before any demo did. The test we wish we had written is a chaos test that kills the primary LLM mid-sentence; failover remains future work.

7. Deployment & Reproducibility

7.1 Docker image

```
docker build -t finalproj .
docker run --env-file .env finalproj \
  python -m researcher ask "What is photosynthesis?" --offline
```

Multi-stage build on `python:3.12.6-slim`, dependencies compiled into a venv in the builder and copied to a slim runtime running as non-root `appuser`. CI builds this image on every pull request. We could not weigh the final image on our machines (no local Docker daemon), which we state plainly; the build log is the proof we have.

7.2 Configuration

LLM_PROVIDER (default `gemini`) selects the synthesis backend; without its key, live synthesis exits with a one-line error. LLM_MODEL names the model. WEB_SEARCH_PROVIDER (default `duckduckgo`, `keyless`) selects web search; Tavily/Serper need their keys. LOG_LEVEL, CACHE_DIR, CACHE_TTL_SECONDS, PER_SOURCE_TIMEOUT_SECONDS, MAX_SOURCES_PER_QUERY, MAX_PARALLEL, RETRY_ATTEMPTS/MIN_WAIT/MAX_WAIT, and DATABASE_URL tune behavior with safe defaults; every one is documented in `.env.example`, and a missing file simply means defaults plus offline mode. One lesson worth recording: the `ai/` package reads raw environment variables, so all three entry points export the project `.env` by absolute path, otherwise running from another directory silently falls back to provider defaults.

8. Limitations & Future Work

- No provider failover: if Gemini is down, synthesis stops with a clean error instead of trying a second backend. This is the first thing we would add with another week.
- Single-process everything: SQLite and the in-memory rate limiter do not span processes, so horizontal scaling needs Postgres plus a shared budget.
- Web search is the fragile leg: DuckDuckGo throttles automated clients by network, so keyless operation depends on Wikipedia and arXiv behaving, with a keyed search API as the fallback.
- Our worst early decision was caching empty hits, fixed late; any similar “cache everything” shortcut would bite the same way.
- We never load-tested beyond the benchmark harness; a busy day with dozens of concurrent questions would queue on the semaphore and the limiter, which is correct but unmeasured.

9. Tools & Acknowledgements

We used AI coding assistance throughout the project the way we would use any productivity tooling: to move faster over routine work instead of spending hours on it. Drafting, boilerplate, and first passes at tests went quicker with it, and we spent the saved time on design, debugging against real provider responses, and verification. Everything in the repository was read, run, tested, and reworked by us where needed, and we stand behind every line in a walk-through.

References

- [1] Google Gemini API documentation, <https://ai.google.dev/docs>
- [2] Tavily search API documentation, <https://docs.tavily.com>
- [3] Wikipedia Action API (opensearch), <https://www.mediawiki.org/wiki/API:Opensearch>
- [4] arXiv API user manual, <https://info.arxiv.org/help/api/user-manual.html>
- [5] `tenacity` retry library, <https://tenacity.readthedocs.io/>
- [6] `httpx` async HTTP client, <https://www.python-httpx.org/>
- [7] `pydantic-settings` documentation, https://docs.pydantic.dev/latest/concepts/pydantic_settings/

A. Appendix — Reproducing the benchmark

Exact commands to reproduce the §3 benchmark. Anyone with the repo and a valid `.env` should be able to run the lines below and get numbers within 20% of the ones in the table.

```
$ git clone https://github.com/nuralnetworks/async-research-assistant && cd async-  
research-assistant  
$ python -m venv .venv && source .venv/bin/activate  
$ pip install -r requirements.txt  
$ cp .env.example .env # keys optional for the simulated benchmark  
$ python scripts/bench.py --offline
```

End of report — thank you for reading.